

Trabalho Prático

Sistema de Gerenciamento de uma Biblioteca

Implementado em Assembly MIPS

Disciplina: Arquitetura de Computadores

Professor: Ederson Naves Fernandes Gonçalves Júnior

Alunos: Gabriel Henrique Aladim e Mateus Von Sperling

Repositório do código-fonte: <https://github.com/MateusVon/Trabalho-Arquitetura-e-Organizacao-de-Computadores.git>

1. Introdução

Este trabalho apresenta o desenvolvimento de um sistema de gerenciamento de biblioteca feito em Assembly MIPS. O objetivo foi aplicar na prática os conceitos de vetores (arrays) e estruturas de repetição vistos em aula.

O programa resolve o problema de controle de acervo de um bibliotecário, permitindo cadastrar livros, registrar empréstimos, processar devoluções e listar o status de tudo o que está mapeado na memória. O sistema funciona baseado em um menu principal em loop, que roda continuamente até que o usuário decida encerrar a execução.

Para organizar os dados de cada livro (código, status e título), utilizamos a lógica de vetores paralelos. Isso significa que a posição X no vetor de códigos corresponde exatamente à mesma posição X nos vetores de status e de títulos.

2. Implementação

Como este é o nosso primeiro contato com a linguagem Assembly MIPS, focamos em uma arquitetura de código simples e linear. Usamos apenas os recursos básicos passados em sala: registradores temporários (\$t0 a \$t7), instruções de desvio condicional (beq, bne, blt, bge, j) para o fluxo e as syscalls padrão de I/O (leitura e impressão de inteiros/strings). Não foram utilizadas funções separadas ou manipulação de pilha (stack); o programa inteiro roda dentro do bloco main usando labels de controle.

2.1. Estrutura de dados utilizada

Os dados ficam alocados na seção .data em três vetores principais:

codigos: Armazena o ID do livro. Para simplificar e evitar erros do usuário, o código é gerado automaticamente e equivale ao próprio índice do vetor (0 para o primeiro, 1 para o segundo, etc.).

status: Guarda um inteiro indicando a situação do livro: 0 para disponível e 1 para emprestado.

titulos: Reserva um espaço de 30 bytes (caracteres) para o nome de cada obra.

A variável qtdLivros monitora o total de obras no sistema. Ela serve tanto para limitar o tamanho do acervo quanto para definir o número do código do próximo livro a ser inserido.

2.2. Como acessamos os vetores

Como o MIPS não possui indexação direta do tipo vetor[i], o cálculo do endereço de memória precisa ser feito manualmente. Os vetores codigos e status usam elementos de 4 bytes (words), exigindo um avanço de 4 em 4 bytes.

O vetor `titulos` avança de 30 em 30 bytes por conta do espaço alocado para as strings. Para realizar essa multiplicação do índice sem usar a instrução de multiplicação direta, implementamos um laço que soma repetidamente o tamanho do dado (4 ou 30) de acordo com o ID do livro. O deslocamento resultante é armazenado em um registrador auxiliar (`indiceInt`) e referenciado via vetor(`registrador`) nas instruções `lw` (load word) e `sw` (store word), seguindo o modelo apresentado nos slides de Arrays da disciplina.

2.3. Funcionamento do menu principal

O ponto de partida do programa exibe as 5 opções disponíveis. A captura da escolha do usuário é avaliada por uma sequência de `beq`. Se o valor bater com uma opção válida, o fluxo salta para a label da funcionalidade; caso contrário, exibe um alerta de opção inválida e retorna ao início. Ao término de qualquer operação, o comando `j loop_menu` força o retorno ao menu principal.

2.4. Cadastrar Livro

O fluxo valida se a biblioteca atingiu o limite de 50 livros comparando `qtdLivros` com o teto. Se houver vaga, o programa captura a string do título e calcula o offset correto. Em seguida, salva o ID automático e define o status inicial como 0 (disponível). O contador `qtdLivros` é incrementado em 1 e o sistema exibe a confirmação do cadastro com o ID gerado.

2.5. Emprestar Livro

O usuário fornece o código do livro. O programa valida se o ID está dentro do intervalo permitido (entre 0 e `qtdLivros - 1`). Passando na validação, o offset é calculado para ler o vetor `status`. Se o valor for 1, o empréstimo é recusado por duplicidade. Se for 0, o registrador muda o valor para 1 e salva de volta na memória via `sw`.

2.6. Devolver Livro

Segue a mesma lógica de busca e validação do empréstimo. A diferença está na checagem do status: o programa só avança se o status atual for 1 (emprestado). Nesse caso, sobrescreve o valor para 0 (disponível). Se o livro já constava como 0, o sistema dispara um erro informando que ele não estava pendente de devolução.

2.7. Listar Livros

Um laço percorre as posições de 0 até `qtdLivros - 1`. A cada iteração, o programa recupera o código e avalia o status para traduzir o 0 ou 1 lógico para as mensagens "Disponível" ou "Emprestado" na tela. Caso `qtdLivros` seja zero, o laço é ignorado e uma mensagem de "Biblioteca Vazia" é exibida.

2.8. Sair

A opção 5 desvia o fluxo para uma mensagem de encerramento e invoca a syscall 10, fechando a execução do simulador.

3. Testes

A- Cadastro de livros

===== BIBLIOTECA =====

1 - Cadastrar Livro

2 - Emprestar Livro

3 - Devolver Livro

4 - Listar Livros

5 - Sair

Opcao: 1

Digite o titulo do livro: Diario de um banana

Diario de um banana

- Codigo 0

Livro cadastrado com sucesso!

===== BIBLIOTECA =====

1 - Cadastrar Livro

2 - Emprestar Livro

3 - Devolver Livro

4 - Listar Livros

5 - Sair

Opcao: 1

Digite o titulo do livro: Diario de um banana 2

Diario de um banana 2

- Codigo 1

Livro cadastrado com sucesso!

===== BIBLIOTECA =====

1 - Cadastrar Livro

2 - Emprestar Livro

3 - Devolver Livro

4 - Listar Livros

5 - Sair

Opcao: 1

Digite o titulo do livro: Diario de um banana 3

Diario de um banana 3

- Código 2

Livro cadastrado com sucesso!

B- Empréstimo de livros

===== BIBLIOTECA =====

1 - Cadastrar Livro

2 - Empréstimo de Livro

3 - Devolver Livro

4 - Listar Livros

5 - Sair

Opção: 2

Digite o código do livro a ser emprestado: 1

Livro emprestado com sucesso!

===== BIBLIOTECA =====

1 - Cadastrar Livro

2 - Empréstimo de Livro

3 - Devolver Livro

4 - Listar Livros

5 - Sair

Opção: 2

Digite o código do livro a ser emprestado: 1

Erro: livro já está emprestado!

C- Devolução de livros

===== BIBLIOTECA =====

1 - Cadastrar Livro

2 - Empréstimo de Livro

3 - Devolver Livro

4 - Listar Livros

5 - Sair

Opção: 3

Digite o código do livro a ser devolvido: 1

Livro devolvido com sucesso!

===== BIBLIOTECA =====

1 - Cadastrar Livro

2 - Emprestar Livro

3 - Devolver Livro

4 - Listar Livros

5 - Sair

Opcao: 3

Digite o codigo do livro a ser devolvido: 1

Erro: livro ja esta disponivel!

===== BIBLIOTECA =====

1 - Cadastrar Livro

2 - Emprestar Livro

3 - Devolver Livro

4 - Listar Livros

5 - Sair

Opcao: 2

Digite o codigo do livro a ser emprestado: 5

Erro: codigo nao existe!

===== BIBLIOTECA =====

1 - Cadastrar Livro

2 - Emprestar Livro

3 - Devolver Livro

4 - Listar Livros

5 - Sair

Opcao: 3

Digite o codigo do livro a ser devolvido: 5

Erro: codigo nao existe!

D- Listagem de livros

===== BIBLIOTECA =====

1 - Cadastrar Livro

2 - Emprestar Livro

3 - Devolver Livro

4 - Listar Livros

5 - Sair

Opcao: 4

Codigo: 0 | Disponivel

Codigo: 1 | Disponivel

Codigo: 2 | Disponivel

===== BIBLIOTECA =====

1 - Cadastrar Livro

2 - Emprestar Livro

3 - Devolver Livro

4 - Listar Livros

5 - Sair

Opcao: 5

Encerrando o sistema...

-- program is finished running --

4. Conclusão

A principal dificuldade no início do projeto foi acertar a aritmética de ponteiros para descobrir o endereço exato dos elementos na memória, principalmente pelo desafio de coordenar três vetores paralelos com tamanhos de bytes diferentes (4 e 30 bytes). Também precisamos lidar com algumas limitações do simulador MARS. Descobrimos na prática que a diretiva `.space` não aceita operações direto na declaração (como `50 * 30`), o que nos obrigou a passar o valor total já calculado (1500). A fase de testes foi essencial para mapear e tratar os cenários de erro (IDs inválidos ou status inconsistentes na devolução/empréstimo), garantindo que o programa rodasse sem quebras ou travamentos. No fim, o trabalho foi fundamental para entender a manipulação de memória em baixo nível e o funcionamento real das syscalls de texto e inteiros.